Singapore University of Technology and Design

50.043 Database Systems

Group 2

SimpleDB - Lab2

Authors	***Student ID***
Jone Chong Jin	1006338
Sarang Nambiar	1006181
Loh Jianyang John	1006360

20/03/2025

Implementation Details:

1. Explain any design decisions you made.

HeapPage.java

For the `insertTuple()` method in HeapPage.java, a for-loop is used to find the first empty slot in the page.

For the `deleteTuple()` and `insertTuple()` methods in HeapPage.java, we decided to abstract the header bit manipulation by implementing the suggested markSlotUsed method.

HeapFile.java

For the `insertTuple()` method in HeapFile.java, a for-loop is used to find the first available page with empty slots.

We made use of the provided `createEmptyPageData` method from `HeapPage` to initialize an empty page for writing tuples if needed, and wrote the page by implementing the writePage method.

BufferPool.java

Instead of using multiple `ConcurrentHashMaps` for storing PageId, page and its last used timestamp, we decided to create a `private class Frame` accessible to `BufferPool` only with attributes like Page and last used timestamp. Hence, using this `Frame` class, we made a `ConcurrentHashMap` to map `PageId` to `Frame`. This gives us the flexibility of adding additional attributes like pincount if needed in the future without needing to make changes in the code to the entire Java file.

For implementing Least Recently Used(LRU) eviction policy, instead of going with an array, we chose to use the `PriorityQueue`(min-heap) object in Java in order to obtain optimize the time complexity of appending a frame(O(log n), where n is the number of items in the `PriorityQueue`) and popping the least recently used framed(O(1)).

Join.java

For Join.java, we used a nested loops join approach for simplicity. We added a private helper function `mergeTuples()` to simplify tuple merging logic. The helper method to merge two tuples ensures that the combined result is correctly formatted.

IntegerAggregator.java

For IntegerAggregator.java, we used a `HashMap` to map group keys to an integer array containing sum, count, min, and max values.

2. Explain the non-trivial part of your code.

Exercise 1:

fetchNext() in Join.java

In Join.java, the `fetchNext()` method required careful handling of two iterators and the currentOuter state to implement the nested loop join correctly. The `mergeTuples()` helper method needed to accurately merge two tuples along with their tuple descriptors, ensuring correctness in the joined output.

Exercise 3 and 4:

markSlotUsed in HeapPage.java

For the bit manipulation in `markSlotUsed`, the `byteIndex` (position of target byte in header) and `bitIndex` (position of target bit in the byte) are both initialized.

Resetting the target bit: A `bitMask` is then created by using the `bitIndex` such that the position of the target bit has a value of 0 and the rest of the positions are 1 to preserve status quo. This `bitMask` is then used in an 'AND' operation with the current bits in the target byte to reset only the target bit to 0. This byte is called '`reset_target_bit`'.

Setting the target bit: A 'used' byte is created where the position of the target bit either has 1 to denote that the target bit slot is used or 0 otherwise, and the other bits are 0. The 'used' byte is then used in an 'OR' operation with '`reset_target_bit`' to set the target bit, not affecting the other bits.

3. Discuss and justify any changes you made to the API.

No changes were made to the original API for the implementation of this task.

4. Describe any missing or incomplete elements of your code.

For exercise 1, the lab only required a simple nested loop for the join operation, but this join operation may not be optimized for large volumes of data.

For exercise 3 and 4, detailed error handling was not implemented.

For `flushPage(PageId page)` function in exercise 5, detailed error handling was not implemented.